

Fast Fourier Transform

Wenyue Zhang

April 29, 2025

Outline

- 1 Fourier Analysis on $\mathbb{Z}(N)$
- 2 Fast Fourier Transform
- 3 Polynomial Multiplication
- 4 Conclusion

The Group $\mathbb{Z}(N)$

Two integers x and y are congruent modulo N if $x - y$ is divisible by N .

The Group $\mathbb{Z}(N)$

Two integers x and y are congruent modulo N if $x - y$ is divisible by N . It can be easily seen that this defines an equivalence relation on $\mathbb{Z}$.

The Group $\mathbb{Z}(N)$

Two integers x and y are congruent modulo N if $x - y$ is divisible by N . It can be easily seen that this defines an equivalence relation on $\mathbb{Z}$. Define $R(x)$ the equivalence class of x , and define

$$R(x) + R(y) = R(x + y).$$

This turns the set of equivalence classes into an abelian group called the group of integers modulo N denoted by $\mathbb{Z}(N)$ or $\mathbb{Z}/N\mathbb{Z}$.

Root of Unity

A complex number z is an N th root of unity if $z^N = 1$. The set of N th roots of unity is

$$\{1, e^{2\pi i/N}, e^{2(2\pi i/N)}, \dots, e^{(n-1)(2\pi i/N)}\}$$

It can be seen that the set forms a group under complex multiplication. Moreover, it is essentially the same as $\mathbb{Z}(N)$.

Root of Unity

A complex number z is an N th root of unity if $z^N = 1$. The set of N th roots of unity is

$$\{1, e^{2\pi i/N}, e^{2(2\pi i/N)}, \dots, e^{(n-1)(2\pi i/N)}\}$$

It can be seen that the set forms a group under complex multiplication. Moreover, it is essentially the same as $\mathbb{Z}(N)$.

We will use $\mathbb{Z}(N)$ to denote either of these two groups.

Fourier Coefficient

Assume $f : [0, 2\pi) \rightarrow \mathbb{C}$ is a 2π -periodic function. The Fourier coefficient of f is defined by

$$a_n = \frac{1}{2\pi} \int_0^{2\pi} f(x) e^{-inx} dx.$$

Fourier Coefficient

Assume $f : [0, 2\pi) \rightarrow \mathbb{C}$ is a 2π -periodic function. The Fourier coefficient of f is defined by

$$a_n = \frac{1}{2\pi} \int_0^{2\pi} f(x) e^{-inx} dx.$$

Under certain conditions, we have

$$f(x) = \sum_{n=-\infty}^{\infty} a_n e^{inx}$$

Fourier Coefficient

Let V be the set of all continuous functions on $[0, 2\pi)$. It is clear that V is a vector space under addition and scalar multiplication.

Fourier Coefficient

Let V be the set of all continuous functions on $[0, 2\pi)$. It is clear that V is a vector space under addition and scalar multiplication. If we define the inner product $\langle f, g \rangle$ by

$$\langle f, g \rangle = \frac{1}{2\pi} \int_0^{2\pi} f \bar{g},$$

then one can verify that $\{e_n\}$ form an orthonormal basis of V , where $e_n(x) = e^{inx}$.

Fourier Coefficient

Now let consider all complex-valued functions on $\mathbb{Z}(N)$. Similarly denote this vector space by V . We define the inner product by

$$\langle f, g \rangle = \sum_{n=0}^{N-1} f(n) \overline{g(n)}.$$

Fourier Coefficient

Now let consider all complex-valued functions on $\mathbb{Z}(N)$. Similarly denote this vector space by V . We define the inner product by

$$\langle f, g \rangle = \sum_{n=0}^{N-1} f(n) \overline{g(n)}.$$

If we define $e_l(k) = e^{2\pi i l k}$ for $l = 0, 1, \dots, N-1$ and $k = 0, 1, \dots, N-1$, then it can be verified that $\{e_0, \dots, e_{N-1}\}$ is an orthogonal basis.

Fourier Coefficient

By results from linear algebra, we can write f as

$$f = \sum_{n=0}^{N-1} a_n e_n,$$

where

$$a_n = \frac{\langle f, e_n \rangle}{\langle e_n, e_n \rangle} = \frac{1}{N} \sum_{k=0}^{N-1} f(k) e^{-2\pi i k n / N}.$$

Fourier Coefficient

By results from linear algebra, we can write f as

$$f = \sum_{n=0}^{N-1} a_n e_n,$$

where

$$a_n = \frac{\langle f, e_n \rangle}{\langle e_n, e_n \rangle} = \frac{1}{N} \sum_{k=0}^{N-1} f(k) e^{-2\pi i k n / N}.$$

In addition,

$$f(k) = \sum_{n=0}^{N-1} a_n e^{2\pi i k n / N}$$

Discrete Fourier Transform

The process to find $a_0, a_1, \dots, a_{N-1}$ is called the discrete Fourier transform.

Discrete Fourier Transform

The process to find $a_0, a_1, \dots, a_{N-1}$ is called the discrete Fourier transform. We can use two loops to compute them, with a time complexity of $O(N^2)$.

Discrete Fourier Transform

The process to find $a_0, a_1, \dots, a_{N-1}$ is called the discrete Fourier transform. We can use two loops to compute them, with a time complexity of $O(N^2)$. We now present the fast Fourier transform, an algorithm that improves the bound $O(N^2)$ to $O(N \log N)$.

Fast Fourier Transform

The algorithm is based on the technique of divide-and-conquer.

Fast Fourier Transform

The algorithm is based on the technique of divide-and-conquer. First we restrict ourselves to the case where $N = 2^n$. After that it is easy to extend the method to all integers by adding 0 term.

Fast Fourier Transform

The algorithm is based on the technique of divide-and-conquer.

First we restrict ourselves to the case where $N = 2^n$. After that it is easy to extend the method to all integers by adding 0 term.

Let $\#(M)$ denote the minimum number of operations needed to calculate all the Fourier coefficients, then it takes $\#(M)$ operations to find $a_0, \dots, a_{M-1}$. The key is contained in the following recursion step.

A Lemma

If we are given $\omega_{2M} = e^{-2\pi i/(2M)}$, then

$$\#(2M) \leq 2\#(M) + 8M$$

A Lemma

If we are given $\omega_{2M} = e^{-2\pi i/(2M)}$, then

$$\#(2M) \leq 2\#(M) + 8M$$

Proof: The calculation of $\omega_{2M}, \dots, \omega_{2M}^{2M}$ requires no more than $2M$ operations. Note that in particular we get $\omega_M = \omega_{2M}^2$. The main idea is that for any given function f on $\mathbb{Z}(M)$, we consider two functions f_0 and f_1 on $\mathbb{Z}(M)$ defined by

$$f_0(r) = f(2r) \text{ and } f_1(r) = f(2r + 1).$$

A Lemma

We assume that it is possible to calculate the Fourier coefficient of f_0 and f_1 in no more than $\#(M)$ operations each. If we denote the Fourier coefficients corresponding to the groups $\mathbb{Z}(2M)$ and $\mathbb{Z}(M)$ by a_k^{2M} and a_k^M , then we have

$$\begin{aligned} a_k^{2M}(f) &= \frac{1}{2M} \sum_{r=0}^{2M-1} f(r) \omega_{2M}^{kr} \\ &= \frac{1}{2} \left(\frac{1}{M} \sum_{r=0}^{M-1} f_0(r) \omega_M^{kr} + \frac{1}{M} \sum_{r=0}^{M-1} f_1(r) \omega_M^{km} \omega_{2M}^k \right). \end{aligned}$$

A Lemma

Therefore

$$a_k^{2M}(f) = \frac{1}{2} (a_k^M(f_0) + a_k^M(f_1)\omega_{2M}^k).$$

As a result, knowing $a_k^M(f_0)$, $a_k^M(f_1)$ and ω_{2M}^k , we see that each $a_k^{2M}(f)$ can be computed using no more than three operations. So

$$\#(2M) \leq 2M + 2\#(M) + 3 \times 2M = 2\#(M) + 8M.$$

Fast Fourier Transform

Then we can prove the bound by induction. The initial step for $n = 1$ is easy, since $N = 2$ and the two Fourier coefficients can be computed easily. Assume $\#(N) \leq 4 \cdot 2^n n$, then by the lemma

$$\#(2N) \leq 2\#(N) + 8N = 2 \cdot 4 \cdot 2^n n + 8 \cdot 2^n = 4 \cdot 2^{n+1}(n + 1).$$

Hence we conclude that $\#(N) = O(N \log N)$.

Polynomial Multiplication

We present one application of fast Fourier transform.

Polynomial Multiplication

We present one application of fast Fourier transform.

Given polynomials F and G , compute their product $F \cdot G$. Assume the degree of F and G are n and m .

Polynomial Multiplication

We present one application of fast Fourier transform.

Given polynomials F and G , compute their product $F \cdot G$. Assume the degree of F and G are n and m .

If we directly compute every term in $F \cdot G$, the time complexity would be $O(nm)$.

Process

First let us extend F and G to a polynomial of degree of N where $N = 2^k$ and $N \geq n + m - 1$ for convenience.

Process

First let us extend F and G to a polynomial of degree of N where $N = 2^k$ and $N \geq n + m - 1$ for convenience.

Now let us get the Fourier coefficients of F and G by fast Fourier transform, thus $F(k) = \sum a_n e^{2\pi i n k / N}$ and $G(k) = \sum b_n e^{2\pi i n k / N}$. Assume the Fourier coefficients of $F \cdot G$ are $\{c_n\}$, then $c_n = a_n b_n$ (why?).

Process

First let us extend F and G to a polynomial of degree of N where $N = 2^k$ and $N \geq n + m - 1$ for convenience.

Now let us get the Fourier coefficients of F and G by fast Fourier transform, thus $F(k) = \sum a_n e^{2\pi i n k / N}$ and $G(k) = \sum b_n e^{2\pi i n k / N}$. Assume the Fourier coefficients of $F \cdot G$ are $\{c_n\}$, then $c_n = a_n b_n$ (why?). Since we know the Fourier coefficients of $F \cdot G$, we surely know $F \cdot G$.

Time Complexity

The time complexity for getting $\{a_n\}$ and $\{b_n\}$ is $O(N \log N)$ by fast Fourier transform.

Time Complexity

The time complexity for getting $\{a_n\}$ and $\{b_n\}$ is $O(N \log N)$ by fast Fourier transform.

It takes $O(N)$ time to compute $\{c_n\}$.

Time Complexity

The time complexity for getting $\{a_n\}$ and $\{b_n\}$ is $O(N \log N)$ by fast Fourier transform.

It takes $O(N)$ time to compute $\{c_n\}$.

The process to get $F \cdot G$ from $\{c_n\}$ is called the inverse fast Fourier transform. It can be done by modifying the algorithm for getting Fourier coefficient and the time complexity is thus $O(N \log N)$.

Time Complexity

The time complexity for getting $\{a_n\}$ and $\{b_n\}$ is $O(N \log N)$ by fast Fourier transform.

It takes $O(N)$ time to compute $\{c_n\}$.

The process to get $F \cdot G$ from $\{c_n\}$ is called the inverse fast Fourier transform. It can be done by modifying the algorithm for getting Fourier coefficient and the time complexity is thus $O(N \log N)$.

The overall time complexity is $O(N \log N)$.

Python Code Implementation

```
1 import numpy as np
2
3 def poly_mul(a, b):
4     n, m = len(a), len(b)
5     L = 1 << (n + m - 1).bit_length()
6     A = np.fft.fft(a, L)
7     B = np.fft.fft(b, L)
8     C = A * B
9     c = np.fft.ifft(C)
10    c = np rint(np.real(c)).astype(int)
11    return c[:n+m-1]
```

Acknowledgements

I am deeply grateful to Frank Cole for his help and guidance, DRP coordinators for organizing this and Elias M. Stein & Rami Shakarchi for their great textbook.

Acknowledgements

I am deeply grateful to Frank Cole for his help and guidance, DRP coordinators for organizing this and Elias M. Stein & Rami Shakarchi for their great textbook.

Thanks for listening.